Certified Rewrite Discovery for Tensor Programs: A Machine-Minted Conv+BN Fold from Minting to Runtime

Draft — proposed, not accepted

mlopt project

2026-10-21

Abstract

Compilers choose better implementations only from transformations a human already wrote. We describe a pipeline in which the rewrite rules themselves are a product of measurement: laws are data, composites are minted by a fixpoint over the shipped library, guards are transported through an evidence store, derivability is credited against an object's own premise closure, and admission is decided by a staged gauntlet that has repeatedly falsified the engine's own candidates. The flagship result is `conv_bn_fold` — the eval-mode batch-norm weight fold a hand optimizer writes — minted, admitted, and shipped verbatim as a machine-authored law in the default rule set. The deployed program evaluates bitwise-equal to the spelled one through an independent runtime and is ~5% cheaper per call in the honest deployed regime — a −16% subprocess arm conflates unused marshal work into the spelled baseline and is quoted only in the regime split — while the term-level cost model prices the fold at −33% on the shipped term (−50% on the probe site). All reported numbers are reproducible across the `handl-lang` trees at the locked revisions (§Section 8). The scope is deliberately honest: a kernel-scale, certified-rewrite result, not a model-scale speedup claim.

> **!** Important
>
> **Status.** This is a working draft — *proposed, not accepted*. It has not had human revision or external review. Every quantitative claim names the probe, test, or retro that produced it; the rest of the paper should be read as a candidate for rewriting, not as a settled result.

1 Introduction

Every optimization a production compiler knows was written by a person — a hand-derived rule, encoded per framework. The rule library is the bottleneck: human authorship is slow, and the pairings nobody names never get found. For a machine to *author* optimizations, four things must exist: rules it can write (data, not code), a referee to check them, a score to rank them, and a world — a runtime — that programs, rules, and derivations all live inside.

The pieces have arrived separately. Massalin's superoptimizer (Massalin 1987) searched programs exhaustively; equality saturation (Tate et al. 2009; Willsey et al. 2021) closes the program space under a fixed law bank, and TENSAT applies it to tensor graphs. Rule-inference work (Ruler (Nandi et al. 2021), the Babble line (Cao et al. 2023)) grows the library itself. And learned play on well-defined games — AlphaGo Zero (Silver et al. 2017), AlphaTensor (Fawzi et al. 2022) — reaches

structure beyond human-written heuristics. The synthesis story for *tensor programs with runtime-priced deployment*, though, is thinner: no system connects discovery, certification, pricing, shipping, and learned guidance inside one substrate.

This paper reports the connection — and one complete measured traversal of it — in the mlopt engine. The contribution is not any single mechanism but their composition: a categorical substrate makes rules data, derivations checkable, cost measurable, and provides the runtime world — which closes discovery into a well-defined game played by learned policies. The enabling property is *self-containedness*: programs, rewrite laws, and derivations are n-morphisms in one calculus — executable values, not external metadata — so the system produces the values it consumes. To our knowledge this is the first *operational* substrate of this kind; categorical languages prior to it are denotational or design-level, not working machines whose runtime executes the morphism hierarchy. The claim is narrow and checkable:

> A rewrite a hand optimizer would write — the eval-mode conv+BN weight fold — was **discovered, certified, priced, and shipped by the machine**, and its deployed form runs bitwise-equal and cheaper through an independent runtime.

No model-scale speedup is claimed. The result is kernel-scale (one op pair, fp64 fixtures), the runtime is a tree-walk interpreter (debug build), and the move alphabet is bounded by the 28-op host-tensor surface — all named in Section 7. What the case study establishes is the *pipeline*: that a law can travel from measured need to a shipped, certified, runtime-priced rule without a human writing it.

The repository is `handl-lang/graphopt`; every number below cites the artifact that produced it (§Section 8).

2 Background

2.1 Equality saturation

An e-graph compactly represents the closure of a term under a law bank — all reachable spellings, sharing subterms, to a fixed point (Tate et al. 2009) — and extraction picks the cheapest member under a cost model. The machinery lives in `handl-lang/eqsat` and the platform paper (mlopt project 2026) describes it once: the term IR, the bounded saturation, the certificate and cost algebra. This paper recounts only what the discovery story uses. One piece does carry over verbatim: the soundness boundary is the law bank — a rule's side conditions are the *only* layer that keeps a false member out, and the certificate replay re-proves that a step is a valid application, not that the rule is true.

2.2 Laws as data on a categorical substrate

Catopt reads a program as a 1-morphism between tensor types, a rewrite law as a 2-cell between programs, and a derivation — one law proven from premises — as a 3-cell: the cells-as-data pattern the kernel calculus defines (handl project 2026) and the eqsat paper operationalises (mlopt project 2026). The consequence is that laws are **data rows** (a pattern pair plus a `cond`/`check`/`derive` guard spec), serializable, storable, composable — and what this paper mutates is the bank: `mlopt_core.laws.ALL_RULES` mixes human-authored axioms with thirteen machine-admitted objects (`mlopt_core.laws.provenance.MACHINE_SHIPPED`).

2.3 The handl substrate and mlopt-kit

The same "one language, many interpretations" model is implemented by the handl platform — the closed arrow calculus and `Language` schema are the kernel paper's subject (handl project 2026): eval, typecheck and rewrite are interpretations of one language whose tables are data. `packages/mlopt-kit` — a plain Rust crate — composes handl's canonical `Language`, its lex/parse/tree-walk-exec engines, and `handl-host-tensor`'s 28-op `tensor.*` surface as dependencies (no forks). `hnl-run` is a subprocess door: `.hnl` in, a JSON wire tensor out, `--repeat N` evaluating a source N times in one process — the measurement hook. `bench/drivers/kit/term_hnl.py` renders any mlopt IR term as a `.hnl` program from a data table (per-op arg assembly, attr marshaling, leaf binding), so the *same object the e-graph rewrites* is what the runtime executes.

3 Machinery

3.1 The mint fixpoint

`mlopt_core.meta.synthesize_rules` runs the substrate's bounded mint fixpoint (mlopt project 2026) over the law bank on a seed term — apply the rules, compose the pairs that chain, emit `syn_*` composites carrying premise provenance. On the eval-mode ConvBN seed it emits 974 rules in ≈ 12 s (fuel 30,000, 4 rounds); the fold composite mints deterministically (identical names across runs).

3.2 Guard transport

A minted rule's side conditions are closures over its premise objects, so the evidence store keeps them as *specs*, not code — parent names plus re-expression maps — and `evidence.admit_object` rebuilds `check`/`derive` by resolving parents transitively; the substrate's shipped form (`mlopt_core.laws.minted`) embeds the records and replays the rebuild at import, refusing a degraded parent (mlopt project 2026).

3.3 Premise-anchored derivability

`verify_law` answers "is this instance derivable under a rule bank?" with a replayable witness, and for a stored composite the honest question — derivability under the premises it was minted from — is the substrate's anti-gaming discipline (mlopt project 2026): the certificate must be standalone-replayable and every step naming a non-axiom rule must *site-verify* under the fp64 numeric oracle. A premise false at its applied site revokes the credit and the verdict falls back to the axioms-only answer.

3.4 One cost alphabet

The substrate's cost model credits compile-time work structurally (mlopt project 2026): a subtree closed over `Param`/`Const` leaves whose root op is in `FOLD_OPS` — the 161-op fold-at-lowering alphabet — is priced at ~0 marginal. This is what lets a fold's pay leg measure what it should: the fold's `mul`/`reshape`/`rsqrt` chain is once-per-deployment work, not per-call work, and `dag_cost` prices it that way.

3.5 The admission gauntlet

`evidence.run_gauntlet` is the domain-side staging of the substrate's admission protocol (mlopt project 2026): reconstruct → full-data → measure → truth → novelty → typed-pay → closure →

certificate, with truth the two-legged numeric-oracle-plus-derivability check and novelty refusing respellings of shipped laws (`relation=duplicate`). What this paper adds is the record of it biting: the gauntlet has repeatedly falsified the engine's own candidates — a naive broadcast guard with 212 bad accepts, an apparent paying sibling with semantically unequal sites, four deliberate false twins (`project/retros/mul-unsq-nat.md`, `layer3-loop.md`, `typed-pay-gate.md`).

3.6 Learned guidance as a game

Because laws are data, discovery admits a well-defined game structure — moves are rule applications, the admission gauntlet is the referee, the cost alphabet is the score (mlopt project 2026). This is implemented machinery, not aspiration, and it lives in `handl-lang/gamekit`: `gamekit.engine` defines the `Board` protocol and the episode/train/eval drivers, and `gamekit.players` provides the `LinearPolicy` arm over a domain-specified featurizer — `mlopt_discovery.engine/players` are compatibility re-export shims over those modules. `mlopt_discovery.play` carries the domain registry (`meta`, `joint`, `torch`, `search`, `gen`, `open`, `player`; `torch` and `gen` ship `deliver` hooks that lower the winning extraction to a verified module). A domain is data — a case generator, a board factory, and a feature schema — not a subclass.

Two linear priors ship with pinned held-out wins: the open-game policy ranks law candidates at ~4.7× the cost-delta heuristic's average precision (AP 0.210 vs 0.045) and ~2× useful laws per episode, and the meta-arena prior beats the scripted playbook (mean episode reward −0.06 vs −0.10) — the latter flagged by its own eval as possibly sampling-variance-riding, an honest negative kept in the record. A distilled contraction policy (`PairPolicyNet`) beats `opt_einsum`'s randomized greedy (0.91× cost at n=40) and ships as a standalone artifact. The neural arm's residual gap is ordering credit, not coverage: a pairwise RankNet objective on the reward's own label reaches AP 0.1941 — above the trained-linear anchor (0.128), under the page-aware linear (0.210) — and a ListNet-graded drain earns heuristic-parity episode reward (0.3549 vs 0.3595, inside seed noise) without beating it (`docs/results_neural_rank.md`). Two learned-policy axes were measured and falsified: learned *schedule guidance* over the generator arms (`project/retros/guide-real-run.md`, `guide-learned-corpus.md` — the optimal schedule reduces to a rule), and *direct law invention* by a learned policy (~10× worse than structured construction; ADR 0004). The evidence for learned guidance is real but early (controlled baseline and ablations in `docs/results_learned_baseline.md` / `docs/results_learned_ablations.md`; dedicated RL arms in `docs/results_learned_rl.md`): linear and distilled arms beat baselines; neural methods reach parity, not wins.

4 Case study: the conv+BN fold

The rewrite is the standard eval-mode fold:

$$\mathrm{BN}_{\gamma,\beta,\mu,\sigma^2,\varepsilon}(\mathrm{conv}(x,W,b)) \;=\; \mathrm{conv}(x,\; W \odot \hat{s}',\; b \odot s + (\beta - s \odot \mu)), \qquad s = \gamma \cdot (\sigma^2+\varepsilon)^{-1/2}.$$

The account below is the probe's own output (`bench/drivers/probes/conv_fold_premise_pay_probe.py`, ~13 s).

Anchor. `CONV_FOLD_RULES` provides the channel-scale and channel-shift anchors (`mul(conv2d(x,W,b),ŝ)`-shaped single-step folds); `BATCH_NORM_RULES` expands `batch_norm` into its affine spelling. Individually they are enabled-but-opt-in — the anchors refuse at truth-starvation, the expand at typed-pay — their value is *enablement*: they are the fold's derivation premises.

Mint. `synthesize_rules` over the bank minus the already-shipped composite emits 974 rules; the fold composite — `batch_norm(conv2d(·),…)` → `conv2d(·)` — mints as a depth-4 `syn_*` chain over an 8-rule transitive premise closure (the `batch_norm_expand` expansion + four `conv_channel_*` anchors + three `syn_*` intermediates).

Transport and truth. Stored with its closure, `admit_object` rebuilds both hooks from guard records (`missing=('check','derive')` → transported). The transported guard accepts the real site (1 eq / 0 ne) and 105/105 of the guarded synth region with 255 honest declines; `verify_law` replays the derivation under shipped axioms in one step — `witness=('conv_bn_fold',)`, the library law proving its own minted origin.

Pay. On the 8×8 probe site the folded instance drops `dag_cost` **34,874.3 → 17,438.7** (−50 %), fires once, verifies `pass` on the lowered modules.

Ship. The composite is promoted *verbatim*: its stored records embed in `mlopt_core.laws.minted`, its guard re-materializes at import, `MACHINE_SHIPPED` grows to 12 (13 today — `add_unsq_pad_r` followed). Re-running the gauntlet post-ship refuses at novelty with `relation=duplicate` — the documented signature of "already admitted."

Runtime-verified. Through the kit bridge (Section 2), the spelled term `batch_norm(conv2d(x,W,b),…)` and the deployed term `conv2d(x,W2,b2)` — with `W2`,`b2` themselves mlopt subterms materialized through the kit — evaluate to the same tensor (**bitwise-equal** on the seeded fixture; the full `torch.export` IR of the module evaluates at 6.7e-16 max abs diff), and per-call wall over `hnl-run --repeat 30` measures three regimes — the honest deployed arm (fold math amortized to load) is ∼−5 %; an inline-fold arm that recomputes the fold per call runs *slower* (+22–33 %); and a subprocess-arm −16 % figure conflates the prelude's unused fold-marshal work into the baseline. `dag_cost` is the stable headline: 55,296 → 36,864 (−33 %). Full reconciliation: `bench/RESULTS.md` + `bench/drivers/probes/conv_fold_wall_regimes_probe.py`.

: The fold decision, priced by the runtime itself — honest regime-split. {#tbl-decision}

One honest subtlety: `dag_cost`'s param-only discount models compile-time folding the tree-walk runtime does not perform, so the measured arm separates fold-once (materialize `W2`,`b2`) from run-every-call (the deployed conv). The cost model and the wall agree in direction; the wall numbers are debug-build timings — directional, not a benchmark rig.

5 Frontier measurements

The discovery loop's own frontier was measured this cycle as a pre-registered question series (`project/TODO.md`). The verdicts sharpen where the open work lives — and it is not where the case study would suggest.

arm	question	measured verdict
Q1	unnamed structure on *real* corpora	**rewrites saturate at known-art; primitives break it** — the rewrite mint over 68 held-out zoo modules admitted 8 paying folds, all **known-art** (already catalogued); the *primitive* arm, which mints new `synop_*` ops for recurring manual shapes no bank op denotes, produced **21/24 admitted machine-original folds** — certified, paying (launch 0.36–0.80x), novel by construction. The honest content: it recovered the shared gated-accumulator skeleton of the SSM family (DeltaNet/GLA/Mamba/RG-LRU) — real cross-architecture structure, not unknown-to-humanity transforms (`docs/reports/primitives_finds.md`)
Q2	rediscovery of a Strassen-class algorithm	**NO-GO** — the 2×2×2 bilinear board provably expresses rank 7 (a scripted 29-move line verifies 10/10), yet random, greedy-repair, and cold-linear players stalled at rank 8 over 200+ episodes at budgets 48 and 96 (`docs/results_strassen.md`)
Q3	multi-step credit — mint a premise, cash it later	**YES, where chains are dense** — the trained linear arm argmaxes the enabling chain on 100% of held-out boards and the neural arm converges to the scripted ceiling; at sparser chain density (more decoys, fewer episodes) linear REINFORCE degrades to the myopic arm (`docs/results_credit_chain.md`)

arm	question	measured verdict
Q4	scale	**three distinct walls** — matcher time on unrolled terms (~11k IR nodes exceeds the 480 s rung budget), verify-forward RAM on the *spelled* program (a ~17 GB fp64 attention tensor at T=4096), and spelled-program numerics (fp64 overflow; verify honestly refuses at T 1024) (`docs/reports/results_scale_ladder.md`)
Q5	prior transfer to a second domain	**null** — a private codegen-domain measurement, cited here without numbers

One substrate bug surfaced en route and is fixed: emit-time RHS sanity conflated *cannot-evaluate* with *evaluated-and-false* through a bare exception catch, so non-denoting composites were being minted as candidates (eqsat `2d6e4eb`). The mint now emits only viable objects — the 101-mint funnel above is the post-fix count.

The pair that teaches. Q2 fails where Q3 succeeds, and the difference is not the machinery — the same board protocol, players, and certifying referee appear in both. It is the shape of the productive trajectory: an enabling chain is a short mint → apply pair the episode stream generates often enough to learn from, while the naive → Strassen path is a ~15-move coordinated sequence through invalid-space intermediates with no positive prefix. **Search depth and chain density — not expressibility, not the referee — are the frontier.**

6 Related work

Equality saturation and e-graph superoptimization. Tate et al. (Tate et al. 2009) introduced the technique; egg (Willsey et al. 2021) made it practical; TENSAT (Yang et al. 2021) and follow-ons apply it to tensor graphs with fixed, hand-written rule sets. Catopt uses the same substrate but the rule bank is not fixed — the discovery loop writes into it through the gauntlet. egglog (Zhang et al. 2023) serves as a test-only differential oracle for the hand-rolled saturation.

Rewrite-rule inference and library learning. Ruler (Nandi et al. 2021) infers rewrite rules for scalar domains by enumeration + equality-saturation filtering — the direct ancestor of the mint-then-verify shape here, differing in that mlopt mints *composites over tensor laws* with transported guards and credits derivability against an object's own premise closure. Babble (Cao et al. 2023) learns library abstractions by anti-unification; mlopt's definitional-atom layer is the same move and measured **null** on reach-expansion — the paying layer is above it.

Generated rule banks. TASO (Jia et al. 2019) auto-generates graph substitutions for DNNs and verifies them formally with an automated theorem prover — the strongest prior on the verify-what-you-mint leg; the generated bank ships once and does not re-enter subsequent discovery. Isaria (Thomas and Bornholt 2024) synthesizes *sound* rewrite-rule systems from ISA specifications to drive e-graph vectorizing compilers, and Enumo (Pal et al. 2023) makes rule inference programmable —

both grow the bank offline, and the ruleset a compiler runs is static. Glenside (Smith et al. 2021) is the closest precedent for machine-derived tensor transformations: e-graphs rediscovering access-pattern rewrites (im2col-style) — the shape of the conv+BN fold here.

Learned play over rewriting. CompilerGym (Cummins et al. 2022) casts compiler optimization as an RL environment over a fixed action space; the Cambridge line — Omelette (Singh 2022), MCTS-GEB (He, Singh, and Yoneki 2023), Aurora (Bărbulescu et al. 2024) — trains RL/MCTS policies that steer e-graph construction. Learned play over rewriting exists, but over a static bank. AlphaGeometry (Trinh et al. 2024) couples a learned proposer to a certifying symbolic deducer — learned play inside a verified loop, over a fixed deduction closure (AlphaTensor (Fawzi et al. 2022) similarly searches one fixed structure).

Each neighbor holds pieces of the loop — generation, certification, learned play — but, to our knowledge, none makes the minted bank itself an object of play: in mlopt a law minted during discovery re-enters as a move inside the learned game (Section 3.6). The differentiation is that closure, not any single leg.

Three capabilities fall out of the closure and belong to no single neighbor. *Two-engine certification*: a certified law must execute identically through mlopt's e-graph *and* through handl's independently-implemented rewrite engine (`packages/mlopt-handl-peer`; the audit driver `bench/drivers/audit/handl_differential.py` fires the structural bank on both and reports disagreement) — a wrong law must fool both. *Laws as first-class values*: minting constructs a `Value` in the same calculus the engine executes — a data row, not generated code — so admission is a runtime decision, not a build step. *Language-agnostic exits*: the `Sink` port lowers one extracted graph to torch (`mlopt_torch.TorchSink`) or ONNX (`mlopt_onnx.OnnxSink`), and a separate bench driver (`bench/drivers/kit/term_hnl.py`) renders the same term as handl `.hnl` — so a certified result is not married to the backend that priced it.

Compilers. TVM (Chen et al. 2018) autotunes schedules; TorchInductor (Ansel et al. 2024) fuses pointwise ops and codegen's the graph a human optimizer could reach. Both are incumbents in the repo's benchmark tables (wins and losses alike); the contribution here is not a faster kernel but a certified, machine-discovered *rewrite*.

Certified/verified compilation. The per-artifact boundary design — law, search, delivery, bound each with its own verifier — is in the spirit of translation validation; what is unusual is that the verifier also audits the *discovery* machinery (adversarial sweeps falsifying minted guards).

7 Limitations

- **Kernel scale.** One op pair, fp64 fixtures, a 1×4×24×24 activation. Nothing here measures model-scale compilation; wider-corpus evidence in the repo (bimodal exposure, ~1.5% yield of fresh paying structure) is honest that this is not yet a general speedup.
- **Move alphabet bound.** The kit surface is 28 `tensor.*` ops (16 forwards + 11 `*_backward` adjoints + `derive`); the fold's own $s = \gamma \cdot \mathrm{rsqrt}(\sigma^2 + \varepsilon)$ is outside it (no elementwise `rsqrt`) and rides a marshaling channel. Emitted terms are confined to what the surface spells; outside-surface ops refuse by name.
- **Tree-walk runtime.** `hnl-run --repeat` timings are wall-clock on a debug-build tree-walk interpreter — they establish direction (the fold's value lands in the deployed program), not competitive throughput. No codegen claim is made.
- **Single-case gauntlet.** The admission measured here is a one-term corpus; the machinery's

broader soundness evidence lives in the library-level sweeps, which caught real holes — but on different objects than this one.

- **Attr-narrow LHS.** The shipped composite's LHS carries the seed's literal attrs (`stride=(1,1)`, `padding=(1,1)`, `eps=1e-05`, `training=False`): its region is the default-attr eval-mode fold — honest, narrow, the common spelling.
- **Novelty is closed-world.** "Novel" in this draft means *machine-original against catalogued knowledge* — the decidable cert ∧ beat ∧ ¬derivable predicate the platform paper defines (mlopt project 2026), evaluated against `known_bank()` — the shipped rules plus the `prior_art` preset's `FUSION/CATEGORICAL` rows (the slice the bounded prover can saturate against) — minus alpha-siblings. The sieve (`bench/drivers/screen/novelty_sieve.py`, results in `docs/results_novelty_sieve.md`) ran 24 candidates over a 242-rule bank: the review queue is **empty** — the two earlier review items (`mul_unsq_nat`, `softsign_fold`) demoted to derivable once their catalogued duals (`mul_unsq_rev`, `softsign_expand`) were registered as prior art — six derivable, six unpaid, twelve unmeasured. "Novel to humanity" remains an open-world literature claim this draft does not make.
- **Scale walls are measured, not hypothetical.** Saturation is T-invariant on symbolic attention terms ($\lesssim 0.3$ s through T=4096), but unrolled spellings hit a matcher wall around ~11k IR nodes (T=1024 exceeds a 480 s budget), the memory wall is the referee's verify-forward of the spelled program (~17 GB at T=4096 — the e-graph had saturated that term in 0.2 s), and an unscaled `exp` attention overflows fp64 at $T \geq 1024$ — verify correctly refuses rather than certify a NaN (`docs/reports/results_scale_ladder.md`).
- **Rewrite-space originality on real corpora is closed; primitive space is open.** The rewrite mint admitted 8 paying folds — every one already a theorem of the registered prior-art bank (known-art). The primitive arm's 21 machine-original folds are novel *by construction* (the `synop_*` op did not exist), which is exactly the falsifiable claim — but the minted shapes are recognizable structure (mul-add trees, broadcast select-gates), not transforms a compiler engineer would call new. The open question is whether a minted primitive can acquire a fused *implementation* the machine also found — the v3 handler grammar (broadcast views admitted into `ew`) now mints verified bodies for 20/24; the 4 refusals are select/linear-bearing shapes — an index-family wall, not budget (`docs/reports/prim_handlers.md`); Strassen-class rediscovery remains out of reach of per-step play (Section 5).

8 Reproduction

Reproducibility is org-scoped, stated honestly. The flagship artifacts — the `conv_bn_fold` shipped law, `MACHINE_SHIPPED` provenance, the `dag_cost` alphabet — live in `mlopt_core` in `handl-lang/eqsat`, and the game layer in `handl-lang/gamekit`; both resolve as private `git+ssh` dependencies pinned in `uv.lock` (eqsat `2ff6fa5…`, gamekit `450731f…`). The measured retros each claim cites live under `project/retros/`, which is gitignored on `main` — they are committed on the repo's `project` branch, not in a `main` clone. So: reproducible across the `handl-lang/{graphopt,eqsat,gamekit}` trees at the locked revisions, requiring org access — not from this repo alone.

Within that scope:

```
uv sync
.venv/bin/python bench/drivers/probes/conv_fold_premise_pay_probe.py   # the admission account
.venv/bin/python -m pytest tests/test_kit_bridge.py -q       # the runtime-priced decision (8 t
cargo test -p mlopt-kit                                     # the Rust consumer (31 tests)
```

```
.venv/bin/python -m pytest -m "not slow"                    # the dev lane
```

The evidence ledger is `bench/RESULTS.md`; the measured retros each claim cites are under `project/retros/` on the `project` branch (notably `minted-composite-default.md`, `kit-term-bridge.md`, `premise-anchored-derivability.md`, `fold-cost-single-source.md`); the extended working draft with the full win/loss tables is `docs/paper_draft.md`.

References

Ansel, Jason, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, et al. 2024. "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation." In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 929–47. ACM. https://doi.org/10.1145/3620665.3640366.

Bărbulescu, George-Octavian, Taiyi Wang, Zak Singh, and Eiko Yoneki. 2024. "Learned Graph Rewriting with Equality Saturation: A New Paradigm in Relational Query Rewrite and Beyond." *arXiv Preprint arXiv:2407.12794*.

Cao, David, Rose Kunkel, Chandrakana Nandi, Max Willsey, Zachary Tatlock, and Nadia Polikarpova. 2023. "Babble: Learning Better Abstractions with e-Graphs and Anti-Unification." *Proceedings of the ACM on Programming Languages* 7 (POPL): 396–424. https://doi.org/10.1145/3571207.

Chen, Tianqi, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, et al. 2018. "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning." In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 578–94. USENIX Association.

Cummins, Chris, Bram Wasti, Jiadong Guo, Brandon Cui, Jason Ansel, Sahir Gomez, Somya Jain, et al. 2022. "CompilerGym: Robust, Performant Compiler Optimization Environments for AI Research." In *Proceedings of the 20th IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 92–105. IEEE. https://doi.org/10.1109/CGO53902.2022.9741258.

Fawzi, Alhussein, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, et al. 2022. "Discovering Faster Matrix Multiplication Algorithms with Reinforcement Learning." *Nature* 610: 47–53.

handl project. 2026. "The Handl Kernel: A Closed Arrow Calculus for Composable Languages."

He, Guoliang, Zak Singh, and Eiko Yoneki. 2023. "MCTS-GEB: Monte Carlo Tree Search Is a Good e-Graph Builder." In *Proceedings of the 3rd Workshop on Machine Learning and Systems (EuroMLSys)*. ACM. https://doi.org/10.1145/3578356.3592577.

Jia, Zhihao, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. "TASO: Optimizing Deep Learning Computation with Automatic Generation of Graph Substitutions." In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, 47–62. ACM. https://doi.org/10.1145/3341301.3359630.

Massalin, Henry. 1987. "Superoptimizer: A Look at the Smallest Program." In *Proceedings of the Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS II)*, 122–26. IEEE Computer Society Press.

mlopt project. 2026. "The Eqsat Substrate: Equality Saturation with Replayable Certificates for Learned Rewrite Games."

Nandi, Chandrakana, Max Willsey, Amy Zhu, Yisu Remy Wang, Brett Saiki, Adam Anderson, Adriana Schulz, Dan Grossman, and Zachary Tatlock. 2021. "Rewrite Rule Inference Using Equality Saturation." *Proceedings of the ACM on Programming Languages* 5 (OOPSLA): 1–28.

https://doi.org/10.1145/3485496.

Pal, Anjali, Brett Saiki, Ryan Tjoa, Cynthia Richey, Amy Zhu, Oliver Flatt, Max Willsey, Zachary Tatlock, and Chandrakana Nandi. 2023. "Equality Saturation Theory Exploration à La Carte." *Proceedings of the ACM on Programming Languages* 7 (OOPSLA2). https://doi.org/10.1145/3622834.

Silver, David, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, et al. 2017. "Mastering the Game of Go Without Human Knowledge." *Nature* 550: 354–59. https://doi.org/10.1038/nature24270.

Singh, Zak. 2022. "Deep Reinforcement Learning for Equality Saturation." Master's thesis, University of Cambridge.

Smith, Gus Henry, Andrew Liu, Steven Lyubomirsky, Scott Davidson, Joseph McMahan, Michael Taylor, Luis Ceze, and Zachary Tatlock. 2021. "Pure Tensor Program Rewriting via Access Patterns." In *Proceedings of Machine Learning and Systems (MLSys)*, 3:273–86.

Tate, Ross, Michael Stepp, Zachary Tatlock, and Sorin Lerner. 2009. "Equality Saturation: A New Approach to Optimization." In *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 264–76. ACM. https://doi.org/10.1145/1480881.1480915.

Thomas, Samuel, and James Bornholt. 2024. "Automatic Generation of Vectorizing Compilers for Customizable Digital Signal Processors." In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 19–34. ACM. https://doi.org/10.1145/3617232.3624873.

Trinh, Trieu H., Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. 2024. "Solving Olympiad Geometry Without Human Demonstrations." *Nature* 625: 476–82. https://doi.org/10.1038/s41586-023-06747-5.

Willsey, Max, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. "Egg: Fast and Extensible Equality Saturation." *Proceedings of the ACM on Programming Languages* 5 (POPL): 1–29. https://doi.org/10.1145/3434304.

Yang, Yichen, Phitchaya Mangpo Phothilimthana, Yisu Remy Wang, Max Willsey, Sudip Roy, and Jacques Pienaar. 2021. "Equality Saturation for Tensor Graph Superoptimization." In *Proceedings of Machine Learning and Systems (MLSys)*. Vol. 3.

Zhang, Yihong, Yisu Remy Wang, Max Willsey, and Zachary Tatlock. 2023. "Better Together: Unifying Datalog and Equality Saturation." *Proceedings of the ACM on Programming Languages* 7 (PLDI): 468–92. https://doi.org/10.1145/3591239.